

T-302-HONN: Project 1

Design Patterns and Design Principles

Þórður Friðriksson
thordurf@ru.is

Háskólinn í Reykjavík — Reykjavík University



Introduction



Attention Please read over all the following points before you start the project

1. Groups

- This assignment can be done in groups of 1-4 people
- Only one member of each group needs to hand in the assignment
- Remember to add each group members name and email to the front page of the handed-in assignment

2. Rules about cheating

- The solution needs to be you own work, if it is discovered that cheating has occurred then all participating parties will receive a 0 for this assignment if this is their first offence on the other hand if this is a repeating occurrence then more severe measures can be expected. See the school's policy and rules on assignments ru.is/namid/reglur/reglur-um-verkefnavinnu

3. Assignment hand-in

- the assignments need to be handed-in on Canvas as a .zip file for the code
- zip file must be named: {student1@ru.is}-{student2@ru.is}-project1.zip
- **If the instructions for the handin are not followed you can expect a grade deduction**

4. Late submissions

- See late submission policy

1 Overview

In this assignment the goal is to build and design a Python logging framework by using the design patterns and principles that we have seen so far.

The motivation behind this project is that students learn how to both build and design software as well as to learn to identify the places in the design and code where design patterns can be applied.

2 Environment

The following requirements are made of the environment you write this project in. You should communicate that environment to the TAs.

- Python 3.6+, State the exact python interpreter version that you are sure that your program runs in.
- Use a virtual environment to reduce the risk of the project failing to run on the machines of the TAs.

3 Structured Logging Framework (70 points)

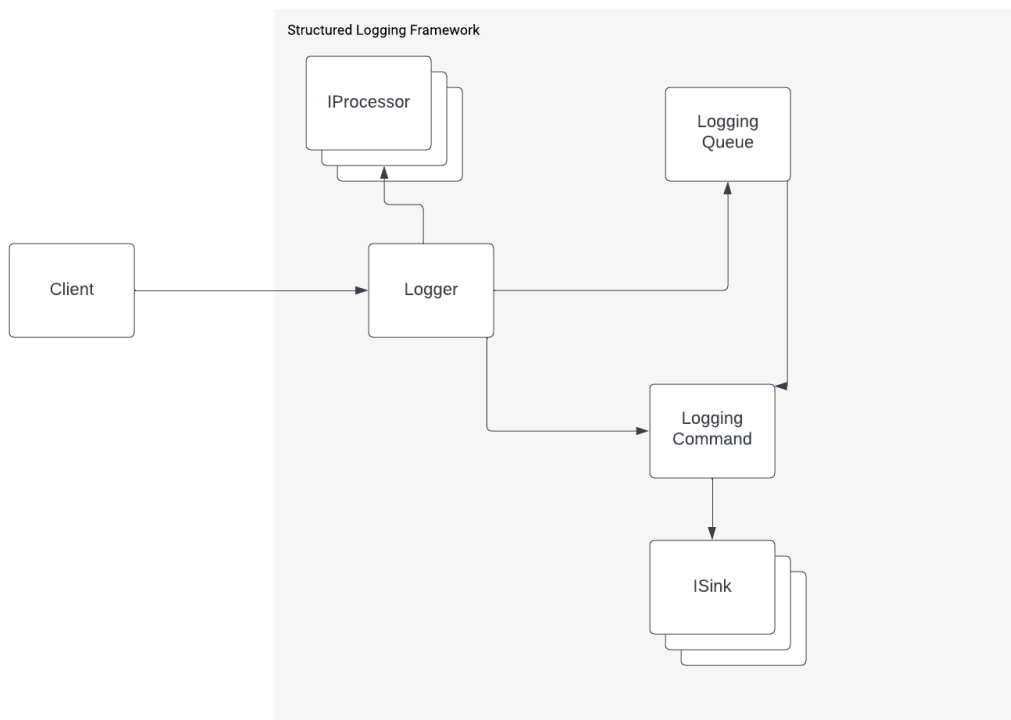
In this part you are supposed to implement the logging framework. This framework should be both simple and extendable.

In short you implement a `Logger` class that has a `log(self, **kwargs: Iterable[Any])` function to log to some specific source.

The logging framework provides implementations for two log sources (**sinks**), those are `ConsoleSink` that logs out to the console and a `FileSink` that logs to a certain file. This should be implemented so that it is easy to add more sinks without doing a lot of changes to existing code.

This logger also has the characteristic of being able to be extended the logging data with so called processors.

The logger is then also supposed to be able to run asynchronously in the background and therefore not have a big performance impact on the user of the framework.



3.1 Sinks (12.5 points)

In this section you should implement `Sinks` these sinks are the ones that take care of logging the data down to some source. Make sure that the internal logic of this sink is hidden from the users of this class. The user should only need to know that this is some kind of sink.

3.1.1 ConsoleSink (5 points)

Should have a function that accepts a dictionary object and writes the results of that object as json to the Console.

3.1.2 FileSink (5 points)

Should have a function that accepts a dictionary object and writes the results of this object as json to a file specified as a file path parameter in the constructor of the `FileSink` class.

3.1.3 What's the pattern? (2.5 points)

The implementation of these sinks is a certain pattern, what is the pattern?



Notice. I recommend that you test these classes before proceeding (Could be a strong move to use `pytest`).

3.2 Processors (25 points)

In this section, you should implement the `Processor` functionality, which is the functionality that runs before the logging logic runs. These processors can then react to the logging data in some way, e.g. by adding to it, changing it, or throwing an error.

These processors should be implemented with the `Chain of Responsibility` pattern, where each processor is executed one after the other.

When implementing the `Chain of Responsibility`, you should also use the `Template Method`

3.2.1 Chain of Responsibility (10 points)

Create an `AbstractProcessor` that encapsulates the chain of responsibility functionality and makes it easy to create other processors by encapsulating a *processing algorithm* with the `Template Method`.

Processors should be able to add functionality for logging. That is, data can f.x. be added before continuing in the chain.

3.2.2 NullProcessor (5 points)

Implement `NullProcessor` which has no function i.e does nothing before or after the chain is continued. This class will be used later by other classes and therefore should not be *used* in this section.

3.2.3 TimestampProcessor (5 points)

Implement `TimestampProcessor` which adds the timestamp as key and the current time as the value to the dictionary.

3.2.4 EnvironmentProcessor (5 points)

Implement an `EnvironmentProcessor` that adds `environment` as key and the specified environment as value. Environment that can be logged can be found in `structured_logging/configuration/environment.py`

3.3 Logger Config Builder (10 points)

In the `configuration` folder in the skeleton code, you can find a `LoggerConfig` model class that defines how the logger should behave.

- **sink** → The sink property specifies which sink the logger will use
- **processor** → Specifies the processing work to be performed for the given logging data.
- **is_async** → Whether the logging function runs synchronously or asynchronously, i.e. in the background or not.
- **async_wait_delay_in_seconds** → Specifies how long to wait in the async function before logging to avoid unnecessary processing when there is nothing to log.

The skeleton for the Builder can be found under `structured_logging/logger_creation/logger_config_builder.py`

The default config from this Builder is:

- **sink** = `ConsoleSink`
- **processor** = `NullProcessor`
- **is_async** = `false`
- **async_wait_delay_in_seconds** = `0`

Finish the implementation of this class.

3.4 Queue (10 points)

In order to be able to make the logging function async when that setting is on, we will use the Command Pattern together with a processing queue to process the log commands in a separate thread.

3.4.1 Command Queue (5 points)

Implement the `Queue` class which can be found under `structured_logging/command_queue`. This class should process a list of `Command` objects in an endless loop. If the list is empty, the queue should wait for `async_wait_delay_in_seconds` before trying again.

3.4.2 Logging Command (5 points)

Encapsulate the logging function in its own `LoggingCommand` object. Create this class in the `structured_logging/logger` folder.

This object is responsible for logging the data using the given sink and data dictionary.

These Command objects are the ones that the Queue will process.

3.5 Logger (7.5 points)

Implement the `Logger` class. A skeleton of this class can be found in `structured_logging/logger/logger.py`

The `log` function does the following:

1. Runs the processing pipeline based on the given processor
2. Creates a `LoggingCommand`
3. if `is_async` is true, it adds it to the queue, otherwise the `Logger` object itself runs the command.

3.6 Dependency Injection (5 points)

Implement dependency injection in a class that can be found under `structured_logging/infrastructure/app_module.py`. This code is then used in `structured_logging/logger_creation/logger_factory.py` where the implementation is given.

4 Client (30 points)

After you've implemented the Logging Framework, it's time to use it. In the Code skeleton you can also find the Client code that has a **very** simple order function. What you need to do is to adapt the Logging Framework to the client code so that the client can start using it.

An example of output when main is run:

Command Line

```
{
  "message": "Order started",
  "level": "info",
  "environment": "production",
  "timestamp": "08/10/2022 13:58:51"
}
{
  "message": "Payment started",
  "level": "info",
  "environment": "production",
  "timestamp": "08/10/2022 13:58:51"
}
{
  "message": "Payment finished",
  "level": "info",
  "environment": "production",
  "timestamp": "08/10/2022 13:58:51"
}
{
  "message": "Order finished",
  "level": "info",
  "environment": "production",
  "timestamp": "08/10/2022 13:58:51"
}
```

4.1 Adaptation (15 points)

4.1.1 implementation (10 points)

Under `client/infrastructure/logging/I_logger.py` you can find the client logger interface that is used throughout the client code. Adapt the client to the logging framework without changing the existing code. Include the new code in `client/infrastructure/logging`

4.1.2 What's the pattern? (2.5 points)

What pattern did you use to implement this functionality?

4.1.3 What's the principle? (2.5 points)

What principle are you adhering to by implementing it like this? Explain.

4.2 LoggerConfigFactory (10 points)

Implement `LoggerConfigFactory` under `client/infrastructure/logging/logging_config_factory.py` based on the settings provided.



Notice. In python, functions are first class citizens (and actually implemented as objects) so currently this *factory* is simply just a function since this function requires no internal state. Since functions are first class citizens in python, we can still compose other classes based on these functions (and can also compose other functions from these functions and work with functional programming). So even though this is not a cluster, we can still contribute to all the principles, patterns and clean code we have seen so far.

4.3 Dependency Injection (5 points)

Wire all the client code together with dependency injection. Implement a class that can be found in `client/infrastructure/dependency_injection/app_module.py`

5 Bonus (15 points)

5.1 Add functionality to be able to log objects (5 points)

Add the functionality to the client logging code (only change the client logger interface and the code you added in 4.1) so that you can also log entire objects passed in.

Then log the payment object in the `PaymentServiceStub` service.

5.2 Masking Processor (10 points)

In `client/infrastructure/logging` create a `MaskingProcessor` that takes in a list of keys to mask. This processor then modifies the dictionary to change the values of these keys to `***`.

After implementing this add the processor to the `LoggingConfig` via the factory and mask sensitive card information (`card_number`, `security_code`).